

Guia de Deploy — CodeHelp CRM em Produção

Sumario

- 1. Visao Geral da Arquitetura
- 2. Pre-requisitos
- 3. Opcao 1: VPS (Recomendado)
- 4. Opcao 2: Docker
- 5. Opcao 3: Plataformas Managed
- 6. Configuracao do Banco de Dados
- 7. Variaveis de Ambiente
- 8. SSL e Seguranca
- 9. Monitoramento e Manutencao
- 10. Troubleshooting

1. Visao Geral da Arquitetura

O CodeHelp CRM e composto por:

- Backend: Node.js + Express + TypeScript (porta 3010)
- Frontend: React + Vite (build estatico)
- Banco: PostgreSQL (producao) / SQLite (desenvolvimento)
- Cache: Redis (opcional mas recomendado)
- WhatsApp: whatsapp-web.js (requer Chromium)

Fluxo em Producao

```
Usuario -> Nginx (porta 80/443) -> Frontend (estatico)
                                     -> Backend API (porta 3010) -> PostgreSQL
                                                                    -> Redis (cache)
                                                                    -> WhatsApp (Chromium)
```

2. Pre-requisitos

Recomendacao de Servidor

Componente	Minimo	Recomendado
CPU	1 vCPU	2 vCPUs
RAM	1 GB	2 GB
Disco	20 GB SSD	40 GB SSD
OS	Ubuntu 22.04	Ubuntu 22.04
Node.js	20.x	20.x LTS

Software Necessario

- Node.js 20.x ou superior

- PostgreSQL 14+
- Redis 6+ (opcional)
- Nginx (para servir frontend + proxy reverso)
- PM2 (gerenciador de processos)
- Chromium (para WhatsApp)
- Git

3. Opcao 1: VPS (Recomendado)

3.1 Provedores Populares

Provedor	Preco Inicial	Observacao
Hetzner	EUR 4.50/mes	Melhor custo-beneficio
DigitalOcean	\$6/mes	Droplet Ubuntu
AWS EC2	Free tier 12m	t3.micro
Vultr	\$5/mes	Boa performance
Linode	\$5/mes	Estavel

3.2 Setup Inicial do Servidor (Ubuntu 22.04)

```
# Conectar no servidor
ssh root@SEU_IP

# Atualizar sistema
apt update && apt upgrade -y

# Instalar Node.js 20.x
curl -fsSL https://deb.nodesource.com/setup_20.x | bash -
apt install -y nodejs

# Verificar versao
node -v
npm -v
```

3.3 Instalar PostgreSQL

```
apt install -y postgresql postgresql-contrib

# Criar banco de dados e usuario
sudo -u postgres psql -c "CREATE USER codehelp WITH PASSWORD 'SUA_SENHA_FORTE';"
sudo -u postgres psql -c "CREATE DATABASE codehelp_db OWNER codehelp;"
sudo -u postgres psql -c "GRANT ALL PRIVILEGES ON DATABASE codehelp_db TO codehelp;"

# Permitir conexoes externas (se necessario)
sed -i "s/#listen_addresses = 'localhost'/listen_addresses = 'localhost'/"
```

```
/etc/postgresql/*/main/postgresql.conf
systemctl restart postgresql
```

3.4 Instalar Redis

```
apt install -y redis-server
systemctl enable redis-server
systemctl start redis-server

# Testar conexao
redis-cli ping # deve retornar PONG
```

3.5 Instalar Nginx

```
apt install -y nginx
systemctl enable nginx
```

3.6 Instalar PM2

```
npm install -g pm2
```

3.7 Instalar Chromium (para WhatsApp)

```
apt install -y chromium-browser
chromium-browser --version
```

3.8 Clonar e Configurar o Projeto

```
# Clonar repositório
cd /opt
git clone https://SEU_USUARIO@SEU_REPO.git code-help
cd code-help

# Instalar dependências
npm install

# Configurar variáveis de ambiente do backend
cd backend
cat > .env << 'EOF'
DATABASE_URL="postgresql://codehelp:SUA_SENHA_FORTE@localhost:5432/codehelp_db"
JWT_SECRET="cole-uma-string-aleatoria-bem-grande-aqui-min-32-char"
JWT_REFRESH_SECRET="cole-outra-string-aleatoria-diferente-aqui-min-32-char"
REDIS_URL="redis://localhost:6379"
PORT=3010
NODE_ENV=production
APP_URL="https://seudominio.com"
```

```
API_URL="https://api.seudominio.com"
SMTP_HOST="smtp.seudominio.com"
SMTP_PORT=587
SMTP_USER="noreply@seudominio.com"
SMTP_PASS="senha_smtp_aqui"
WHATSAPP_CHROME_PATH="/usr/bin/chromium-browser"
WHATSAPP_SESSION_PATH="/opt/code-help/backend/whatsapp-session"
TIMEZONE="America/Fortaleza"
EOF

# Gerar strings aleatorias seguras
# openssl rand -hex 32
```

3.9 Build e Migracao

```
cd /opt/code-help

# Build do projeto
npm run build

# Rodar migrations do Prisma
cd backend
npx prisma migrate deploy

# Executar seed (cria usuario admin padrao)
npx prisma db seed

cd /opt/code-help
```

3.10 Configurar Nginx

```
cat > /etc/nginx/sites-available/codehelp << 'NGINX'
server {
    listen 80;
    server_nameseudominio.comwww.seudominio.com;

    # Frontend (arquivos estaticos do React)
    location / {
        root /opt/code-help/frontend/dist;
        try_files $uri $uri/ /index.html;

        location ~* \.(js|css|png|jpg|jpeg|gif|ico|svg|woff|woff2|ttf|eot)$ {
            expires 1y;
            add_header Cache-Control "public, immutable";
        }
    }

    # API Backend (proxy reverso)
    location /api/ {
```

```

        proxy_pass http://127.0.0.1:3010;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_cache_bypass $http_upgrade;
        proxy_read_timeout 300s;
        proxy_send_timeout 300s;
    }

    # Storage (uploads)
    location /storage/ {
        proxy_pass http://127.0.0.1:3010;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }

    client_max_body_size 50M;

    gzip on;
    gzip_types text/plain text/css application/json application/javascript text/xml
    application/xml text/javascript;
    gzip_min_length 256;
}
NGINX

ln -sf /etc/nginx/sites-available/codehelp /etc/nginx/sites-enabled/
rm -f /etc/nginx/sites-enabled/default

nginx -t
systemctl reload nginx

```

3.11 Iniciar o Backend com PM2

```

cd /opt/code-help

pm2 start backend/dist/server.js --name "codehelp-api" --max-memory-restart 300M
pm2 save
pm2 startup
# Executar o comando que o PM2 exibir

```

3.12 Verificar Funcionamento

```

pm2 status
pm2 logs codehelp-api

```

```
curl http://localhost:3010/api/health
curl http://localhost/api/health
```

4. Opcao 2: Docker

4.1 Dockerfile

Crie o arquivo `Dockerfile` na raiz do projeto:

```
# Build stage - Frontend
FROM node:20-alpine AS frontend-builder
WORKDIR /app/frontend
COPY frontend/package*.json ./
RUN npm ci
COPY frontend/ ./
RUN npm run build

# Build stage - Backend
FROM node:20-alpine AS backend-builder
WORKDIR /app/backend
COPY backend/package*.json ./
RUN npm ci
COPY backend/ ./
RUN npx prisma generate
RUN npm run build

# Runtime
FROM node:20-alpine

RUN apk add --no-cache chromium nss freetype harfbuzz ca-certificates ttf-freefont

ENV PUPPETEER_SKIP_CHROMIUM_DOWNLOAD=true
ENV CHROMIUM_BIN=/usr/bin/chromium-browser
ENV NODE_ENV=production

WORKDIR /app

COPY --from=backend-builder /app/backend/dist ./backend/dist
COPY --from=backend-builder /app/backend/node_modules ./backend/node_modules
COPY --from=backend-builder /app/backend/package.json ./backend/
COPY --from=backend-builder /app/backend/prisma ./backend/prisma

COPY --from=frontend-builder /app/frontend/dist ./frontend/dist

RUN mkdir -p /app/backend/storage/pdfs /app/backend/whatsapp-session

WORKDIR /app/backend
EXPOSE 3010

CMD ["sh", "-c", "npx prisma migrate deploy && node dist/server.js"]
```

4.2 docker-compose.yml

```
version: '3.8'

services:
  db:
    image: postgres:16-alpine
    container_name: codehelp-db
    environment:
      POSTGRES_USER: codehelp
      POSTGRES_PASSWORD: ${DB_PASSWORD:-codehelp123}
      POSTGRES_DB: codehelp_db
    volumes:
      - pgdata:/var/lib/postgresql/data
    ports:
      - "127.0.0.1:5432:5432"
    restart: unless-stopped
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U codehelp"]
      interval: 10s
      timeout: 5s
      retries: 5

  redis:
    image: redis:7-alpine
    container_name: codehelp-redis
    restart: unless-stopped

  api:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: codehelp-api
    env_file: ./backend/.env
    ports:
      - "127.0.0.1:3010:3010"
    depends_on:
      db:
        condition: service_healthy
      redis:
        condition: service_started
    volumes:
      - ./backend/storage:/app/backend/storage
      - ./backend/whatsapp-session:/app/backend/whatsapp-session
    restart: unless-stopped

  nginx:
    image: nginx:alpine
    container_name: codehelp-nginx
    ports:
      - "80:80"
```

```

    - "443:443"
volumes:
    - ./frontend/dist:/usr/share/nginx/html:ro
    - ./nginx-docker.conf:/etc/nginx/conf.d/default.conf:ro
depends_on:
    - api
restart: unless-stopped

volumes:
    pgdata:

```

4.3 nginx-docker.conf

```

server {
    listen 80;
    server_name _;

    location / {
        root /usr/share/nginx/html;
        try_files $uri $uri/ /index.html;
    }

    location /api/ {
        proxy_pass http://api:3010;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_cache_bypass $http_upgrade;
    }

    location /storage/ {
        proxy_pass http://api:3010;
        proxy_set_header Host $host;
    }

    client_max_body_size 50M;
}

```

4.4 Iniciar com Docker

```

# Criar arquivo .env na raiz
cat > .env << 'EOF'
DB_PASSWORD=SUA_SENHA_FORTE_AQUI
EOF

```



```
# Build e iniciar
docker compose up -d --build

# Verificar status
docker compose ps

# Logs
docker compose logs -f api
```

5. Opcao 3: Plataformas Managed

5.1 Railway (railway.app)

1. Criar conta em railway.app
2. Conectar repositório GitHub
3. Adicionar serviço PostgreSQL (plugin)
4. Configurar variáveis de ambiente
5. Deploy automático a cada push

Variáveis obrigatórias no Railway:

```
DATABASE_URL=<pega do plugin PostgreSQL>
JWT_SECRET=<gerar aleatorio>
JWT_REFRESH_SECRET=<gerar aleatorio>
NODE_ENV=production
APP_URL=https://seudominio.up.railway.app
API_URL=https://seudominio.up.railway.app
```

Limite: 500h/mes no free tier.

5.2 Render (render.com)

1. Criar conta em render.com
2. Web Service -> conectar GitHub
3. PostgreSQL -> criar banco
4. Build command: `npm install && npm run build`
5. Start command: `cd backend && npx prisma migrate deploy && node dist/server.js`

Limite: 750h/mes no free tier.

5.3 Fly.io

1. Instalar flyctl: `curl -L https://fly.io/install.sh | sh`
 2. `fly launch` na raiz do projeto
 3. `fly deploy`
 4. Adicionar volume persistente para WhatsApp session
-

6. Configuracao do Banco de Dados

6.1 Criar Banco (PostgreSQL)

```
-- Conectar como superuser
sudo -u postgres psql

-- Criar usuario
CREATE USER codehelp WITH PASSWORD 'senha_forte';

-- Criar banco
CREATE DATABASE codehelp_db OWNER codehelp;

-- Conceder privilegios
GRANT ALL PRIVILEGES ON DATABASE codehelp_db TO codehelp;

-- Sair
\q
```

6.2 Rodar Migrations

```
cd /opt/code-help/backend

# Aplicar migrations
npx prisma migrate deploy

# Criar seed padrao
npx prisma db seed
```

6.3 Usuario Padrao (Seed)

Email	Senha	Role
admin@codemed.com.br	admin123	admin
vendedor@codemed.com.br	admin123	vendedor

Alterar a senha do admin apos primeiro login!

6.4 Backup Automatico

Crontab para backup diario:

```
# Editar crontab
crontab -e

# Adicionar linha (backup as 3h da manha)
0 3 * * * pg_dump -U codehelp codehelp_db | gzip > /opt/code-help/backups/codehelp_$(date
+%\Y%\m%\d).sql.gz

# Criar pasta de backups
mkdir -p /opt/code-help/backups
```

Para restaurar:

```
zcat /opt/code-help/backups/codehelp_20260618.sql.gz | sudo -u postgres psql codehelp_db
```

7. Variaveis de Ambiente

Arquivo .env do Backend

```
# === BANCO DE DADOS ===
DATABASE_URL="postgresql://codehelp:SENHA@localhost:5432/codehelp_db"

# === JWT (gerar strings aleatorias de pelo menos 32 caracteres) ===
JWT_SECRET="minha-string-secreta-jwt-bem-grande-32chars-min"
JWT_REFRESH_SECRET="minha-string-refresh-secreta-tambem-grande"

# === SERVIDOR ===
PORT=3010
NODE_ENV=production

# === URLs ===
APP_URL="https://seudominio.com"
API_URL="https://api.seudominio.com"

# === REDIS (opcional) ===
REDIS_URL="redis://localhost:6379"

# === WHATSAPP ===
WHATSAPP_CHROME_PATH="/usr/bin/chromium-browser"
WHATSAPP_SESSION_PATH="/opt/code-help/backend/whatsapp-session"

# === EMAIL (SMTP) ===
SMTP_HOST="smtp.seudominio.com"
SMTP_PORT=587
SMTP_USER="noreply@seudominio.com"
SMTP_PASS="senha_smtp"

# === TIMEZONE ===
TIMEZONE="America/Fortaleza"

# === WHATSAPP ALERTAS (opcional) ===
ALERT_WHATSAPP_NUMBERS="+5511999999999"
ALERT_DAY=1
ALERT_HOUR=8
```

Gerar Strings Aleatorias Seguras

```
# No terminal Linux/Mac
openssl rand -hex 32
openssl rand -hex 32
```

```
# Ou no Node.js
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

8. SSL e Seguranca

8.1 Certificado com Let's Encrypt

```
# Instalar Certbot
apt install -y certbot python3-certbot-nginx

# Obter certificado (substituir dominio)
certbot --nginx -d seudominio.com -d api.seudominio.com

# Auto-renovacao (ja configurada pelo certbot)
certbot renew --dry-run
```

8.2 Firewall (UFW)

```
# Ativar firewall
ufw allow OpenSSH
ufw allow 'Nginx Full'
ufw enable

# Status
ufw status
```

8.3 Checklist de Seguranca

Item	Status
JWT_SECRET com +32 caracteres	☐
JWT_REFRESH_SECRET diferente do JWT_SECRET	☐
Senha do banco forte	☐
Senha do admin alterada apos seed	☐
HTTPS ativo	☐
Firewall configurado	☐
Rate limit no login (ja configurado)	☐
CORS configurado para dominio especifico	☐
.env no .gitignore	☐
Node.js atualizado	☐

9. Monitoramento e Manutencao

9.1 Comandos PM2

```
# Status
pm2 status

# Logs tempo real
pm2 logs codehelp-api

# Monitorar recursos
pm2 monit

# Reiniciar
pm2 restart codehelp-api

# Parar
pm2 stop codehelp-api

# Atualizar apos deploy git
cd /opt/code-help
git pull
npm install
npm run build
cd backend && npx prisma migrate deploy
pm2 restart codehelp-api
```

9.2 Health Check

```
# Verificar se a API esta respondendo
curl https://seudominio.com/api/health

# Resposta esperada:
# {"status":"ok","timestamp":"2026-06-18T..."}
```

9.3 Atualizacao do Sistema

```
cd /opt/code-help

# 1. Backup do banco
pg_dump -U codehelp codehelp_db > /tmp/backup_$(date +%Y%m%d).sql

# 2. Pull das alteracoes
git pull

# 3. Instalar novas dependencias
npm install
```

```
# 4. Build
npm run build

# 5. Migrations (se houver)
cd backend && npx prisma migrate deploy

# 6. Reiniciar API
pm2 restart codehelp-api

# 7. Verificar
curl https://seudominio.com/api/health
```

10. Troubleshooting

Erro: "Cannot find module" apos build

```
# Reinstalar dependencias
rm -rf node_modules
npm install
npm run build
```

Erro: "EACCES permissao negada" na pasta whatsapp-session

```
chmod -R 755 /opt/code-help/backend/whatsapp-session
chown -R node:node /opt/code-help/backend/whatsapp-session
```

Erro: "Connection refused" no banco

```
# Verificar se PostgreSQL esta rodando
systemctl status postgresql

# Reiniciar se necessario
systemctl restart postgresql

# Testar conexao
psql -U codehelp -d codehelp_db -h localhost
```

Erro: "EADDRINUSE" porta 3010 em uso

```
# Encontrar processo usando a porta
lsof -i :3010

# Matar processo
kill -9 <PID>
```

```
# Ou usar porta diferente
PORT=3011 pm2 start backend/dist/server.js --name codehelp-api
```

Erro: WhatsApp nao conecta

```
# Verificar se Chromium esta instalado
which chromium-browser

# Verificar se a pasta de sessao existe
ls -la /opt/code-help/backend/whatsapp-session/

# Resetar sessao WhatsApp (apagar pasta e reconectar)
rm -rf /opt/code-help/backend/whatsapp-session/*
pm2 restart codehelp-api
```

Logs do PM2

```
# Todos os logs
pm2 logs codehelp-api

# Apenas erros
pm2 logs codehelp-api --err

# Ultimas 100 linhas
pm2 logs codehelp-api --lines 100

# Salvar logs em arquivo
pm2 logs codehelp-api --lines 1000 > /var/log/codehelp.log
```

Performance Baixa

```
# Verificar uso de memoria
pm2 monit

# Aumentar limite de memoria
pm2 restart codehelp-api --max-memory-restart 500M

# Verificar uso do disco
df -h

# Verificar processos
top -u node
```

Comando Rapido (Resumo VPS)

Apos configurar o servidor e o .env:

```
cd /opt/code-help  
npm install && npm run build  
cd backend && npx prisma migrate deploy && npx prisma db seed  
pm2 start backend/dist/server.js --name codehelp-api  
pm2 save  
pm2 startup
```

Acessar: `http://SEU_IP` ou `https://seudominio.com`